Arbeitsgruppe Genominformatik
Stefan Kurtz

Universität Hamburg
Zentrum für Bioinformatik

Fragen zur Vorbereitung auf die Prüfung im Modul Grundlagen der Sequenzanalyse 30. Januar 2023

Die folgenden Fragen sollen Ihnen bei der Vorbereitung auf die Abschlussklausur im Modul GSA helfen.

1 Das Modell der Edit-Distanz

1.1. Warum ist der Begriff der Ähnlichkeit bei der Analyse biologischer Sequenzen so wichtig?

1.2. Welches Distanzmodell wird in der Sequenzanalyse am häufigsten verwendet?

1.3. Definieren Sie präzise die drei Begriffe *Edit-Operation*, *Alignment* und *Alignment von zwei Sequenzen u und v*.

1.4. Geben Sie die in der Vorlesung entwickelte Rekurrenz an, um die *Anzahl* aller Alignments zweier Sequenzen der Länge m und n zu berechnen.

1.5. Wie lässt sich die Menge *aller* Alignments zweier Sequenzen der Längen m und n in $O(mn)$ Speicherplatz implizit repräsentieren?

1.6. Mit welchen Datenstrukturen kann man ein einziges Alignment effizient repräsentieren? Blatt 9

1.7. Was ist eine Kostenfunktion im Kontext des Edit-Distanz Modells?

1.8. Wie berechnet man mit einer Kostenfunktion δ, definiert für alle Edit-Operationen, die Kosten $\delta(A)$ eines gegebenen Alignments A? Setzen Sie dabei das lineare Gap-Kostenmodell voraus und geben Sie eine entsprechende Gleichung an.

1.9. Sei δ eine Kostenfunktion und seien u und v zwei Sequenzen. Definieren Sie $edist_\delta(u, v)$, d.h. die Edit-Distanz von u und v. Hier ist keine Rekurrenz gefragt.

1.10. Definieren Sie den Begriff *optimales Alignment von u und v*. Sie können dabei den Begriff der Edit-Distanz voraussetzen.

1.11. Sei δ eine Kostenfunktion und seien u und v zwei Sequenzen der Längen m bzw. n. In der Vorlesung wurde die Edit-Distanz Matrix E_δ eingeführt. Welche Eigenschaft hat $E_\delta(i, j)$ für alle i, j mit $0 \leq i \leq m$ und $0 \leq j \leq n$. Hier ist nach der Definition von $E_\delta(i, j)$ gefragt, aber nicht nach der Rekurrenz.

1.12. Sei δ eine Kostenfunktion und seien u und v zwei Sequenzen der Längen m bzw. n. In der Vorlesung wurde die Edit-Distanz Matrix E_δ eingeführt. Für alle i, j mit $0 \leq i \leq m$

und $0 \leq j \leq n$ gilt laut Definition $E_\delta(i,j) = edist_\delta(u[1\ldots i], v[1\ldots j])$. Geben Sie die Rekurrenz für $E_\delta(i,j)$ an, wenn $i > 0$ und $j > 0$ ist.

1.13. Es soll die Edit-Distanz $edist_\delta(u,v)$ für die Sequenzen $u = gcact$ und $v = tgatat$ berechnet werden. Dabei soll δ die Einheitskostenfunktion sein. Für die Berechnung wird die Edit-Distanz Matrix berechnet, die (bis auf die letzte Spalte) unten angegeben ist. Dabei sind in der zweiten Zeile und zweiten Spalte die Spalten- bzw. Zeilenindizes angegeben. Berechnen Sie die Werte in der letzten Spalte und tragen Sie diese in die Matrix ein. Kennzeichnen Sie den Wert in der Matrix, aus dem sich $edist_\delta(u,v)$ ergibt.

R > D > I

			t	g	a	t	a	t
	$E_\delta(i,j)$	0	1	2	3	4	5	6
	0	0	1	2	3	4	5	
g	1	1	1	1	2	3	4	
c	2	2	2	2	2	3	4	
a	3	3	3	3	2	3	3	
c	4	4	4	4	3	3	4	
t	5	5	4	5	4	3	4	

1.14. Welche Laufzeit und welchen Speicherplatzbedarf hat das in der Vorlesung vorgestellte Verfahren zur Berechnung der Edit-Distanz von zwei Sequenzen u und v der Längen m und n? Hier geht es um das „distance-only" Verfahren, das nur die Edit-Distanz, jedoch kein Aligment bestimmt. Verwenden Sie die O-Notation.

1.15. Definieren Sie den Begriff *Edit-Graph* für zwei Sequenzen u und v der Längen m und n. Aus welchen Knoten und aus welchen Kanten besteht der Graph? Sind Knoten und Kanten markiert, und wenn ja, womit?

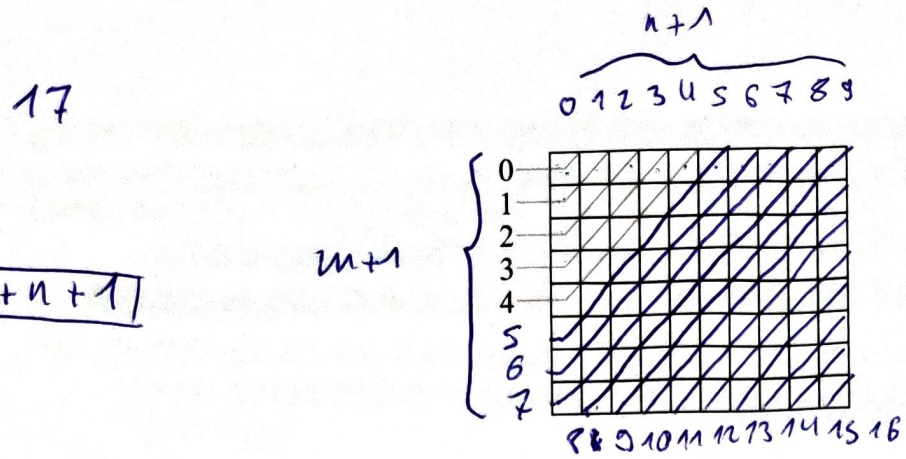
1.16. Welches Verfahren wird angewendet, um die optimalen globalen Alignments für zwei Sequenzen u und v der Länge m bzw. n aus der Matrix E_δ zu bestimmen? Beschreiben Sie dieses Verfahren kurz und präzise in höchstens drei Sätzen.

1.17. Definieren Sie in einem Satz den Begriff *Graph*.

1.18. Sei $G = (V,E)$ ein Graph. Definieren Sie den Begriff *Pfad in G*.

1.19. Sei $G = (V,E)$ ein gerichteter Graph. Definieren Sie den Begriff der topologischen Ordnung $\prec$ der Knoten aus V.

1.20. Sei E_δ die Matrix, die zur Lösung des Editdistanzproblems für zwei Sequenzen u und v der Länge m bzw. n berechnet wird. Anstatt einer spalten- oder zeilenweisen Berechnung der Werte in E_δ können diese auch antidiagonalweise berechnet werden. Eine Antidiagonale umfasst Werte auf einer Linie von Südwest nach Nordost, wie in der folgenden Abbildung für die Antidiagonalen 0 bis 4 dargestellt:



Beantworten Sie die folgenden Fragen:

1.20.1. Wieviele Antidiagonalen gibt es in E_δ in Abhängigkeit von m und n?

1.20.2. Geben Sie die Eigenschaft aller Paare (i, j) an, die zu der gleichen Antidiagonalen a gehören.

1.20.3. Von welchen Antidiagonalen in E_δ hängen die Werte der Antidiagonale a für $a \geq 2$ ab?

2 Varianten des Edist-Distanz Modells

2.1. Sei σ eine Score-Funktion, und seien u und v zwei Sequenzen der Längen m bzw. n. Zur Bestimmung optimaler lokaler Alignments von u und v wird eine $(m + 1) \times (n + 1)$-Matrix L_σ berechnet. Welche Eigenschaft hat $L_\sigma(i, j)$ für alle i, j mit $0 \leq i \leq m$ und $0 \leq j \leq n$. Hier ist nicht nach der Rekurrenz gefragt. Die Beschreibung der Eigenschaft soll vollständig sein. Sie können aber die Begriffe Alignment und $\sigma(A)$ für ein Alignment A vorraussetzen.

2.2. Sei σ eine Score-Funktion und seien u und v zwei Sequenzen der Längen m bzw. n. Zur Bestimmung optimaler lokaler Alignments von u und v wird eine $(m + 1) \times (n + 1)$-Matrix L_σ berechnet. Geben Sie die Rekurrenz für $L_\sigma(i, j)$ an für den Fall, dass $i > 0$ und $j > 0$.

2.3. Im Kontext des Smith-Waterman Algorithmus wurde die Matrix L_σ eingeführt. Wir nehmen nun an, dass diese Matrix für zwei Sequenzen u und v bereits berechnet wurde. Welches Verfahren wird angewendet, um ein optimales lokales Alignment von u und v mit Hilfe der Matrix L_σ zu bestimmen? Beschreiben Sie dieses Verfahren kurz und präzise in höchstens drei Sätzen.

2.4. Sei $k \geq 0$ eine Fehlerschranke. Was ist ein approximativer Treffer eines Musters p in einem Text t? Sie können dabei die Definition der Edit-Distanz vorraussetzen.

2.5. Sei $k \geq 0$ eine Fehlerschranke. Seien p und t Sequenzen der Längen m bzw. n. Wie kann man die Endpositionen aller approximativen Treffer von p in t berechnen, wenn p die Länge m und t die Länge n hat? Welche Laufzeit hat dieses Verfahren? Wie unterscheidet sich die zu berechnende Matrix D_δ von der Edit-Distanz Matrix E_δ hinsichtlich der Matrixeinträge in der ersten Zeile?

2.6. Sei $k \geq 0$ eine Fehlerschranke. Seien p und t Sequenzen der Längen m bzw. n, so dass $m \ll n$, d.h. m ist wesentlich kleiner als n. Ein approximativer Treffer ist ein Substring w von t, so dass $edist_\delta(w, p) \leq k$. Wie lang kann ein approximativer Treffer höchstens werden, wenn δ die Einheitskostenfunktion ist? Begründen Sie Ihre Antwort.

2.7. Was versteht man unter dem Überlappungsproblem bei der Sequenzassemblierung?

3 Spezielle Gapkosten

3.1. Zu Beginn der Vorlesung wurde das Modell der linearen Gapkosten eingeführt. Wie lässt sich dieses Modell zum allgemeinen Gap-Kostenmodell verallgemeinern?

3.2. Seien u und v zwei Sequenzen der Längen m und n. In welcher Laufzeit kann man für das allgemeine Gap-Kostenmodell die Edit-Distanz bestimmen?

3.3. Definieren Sie den Begriff der affinen Gap-Kosten.

4 Sequenzvergleich ohne Alignments

4.1. Seien u und v zwei Sequenzen der Längen m bzw. n. Definieren Sie den Begriff der *Zerlegung von v bzgl. u*. Definieren Sie die Maximal-Matches-Distanz von v und u.

4.2. In der Vorlesung wurde das Maximal-Matches-Modell für den paarweisen Vergleich von Sequenzen betrachtet.

4.2.1. Geben Sie ein Beispiel für ein Paar von Sequenzen an, das zeigt, dass die Maximal-Matches-Distanz keine Metrik ist. Begründen Sie Ihre Antwort.

4.2.2. Definieren Sie, wie in der Vorlesung, auf der Basis der Maximal-Matches-Distanz eine Metrik.

4.3. Mit welcher Laufzeit und Speicherplatz-Effizienz läst sich nach dem Maximal-Matches-Modell die Distanz $mmdist(v, u)$ zwischen zwei Sequenzen v und u berechnen?

4.4. In welcher Relation steht die Maximal-Matches Distanz zur Edit-Distanz bzgl. der Einheitskostenfunktion? Wie kann man diese Relation nutzen?

4.5. Seien u und v zwei Sequenzen der Längen m bzw. n über einem Alphabet der Größe r. Sei $q > 0$.

4.5.1. Definieren Sie den Begriff des q-Wort Profils von u.

4.5.2. Definieren Sie auf der Basis der q-Wort Profil von u und von v die q-Wort Distanz von u und v.

4.6. Sei $q \geq 2$. Geben Sie ein Beispiel für zwei Sequenzen an, das zeigt, dass die q-Wort-Distanz keine Metrik ist.

4.7. Beschreiben Sie präzise, wie man q-Worte über einem Alphabet der Größe r durch ganze

Zahlen im Wertebereich von 0 bis $r^q - 1$ codieren kann.

4.8. In der Vorlesung wurde gezeigt, wie man q-Worte über einem Alphabet der Größe r durch ganze Zahlen im Wertebereich von 0 bis $r^q - 1$ codieren kann. Welche Eigenschaften dieser Codierung nutzt man aus, um nacheinander die Codes aller q-Worte einer Sequenz effizient zu bestimmen?

4.9. Mit welcher Laufzeit und welchem Speicherplatzverbrauch kann man die q-Wort Distanz für zwei Sequenzen der Länge m bzw. n über einem Alphabet der Größe r berechnen?

4.10. Sei $q \geq 1$ und seien u und v zwei Sequenzen der Länge m bzw. n, jeweils über einem Alphabet $\mathcal{A}$ der Größe r. Beschreiben Sie einen Algorithmus, der in $O(m+n+r^q)$ Zeit und $O(r^q)$ Platz die gesamte Anzahl der Paare (i, j) berechnet, so dass gilt:

- $1 \leq i \leq m - q + 1$ und $1 \leq j \leq n - 1 + 1$,
- $u[i \ldots i + q - 1] = v[j \ldots j + q - 1]$

Jeder solche Paar repräsentiert also einen Treffer eines q-Wortes. Beispiel: Für $q = 2$ und die Sequenzen $u = aaca$, $v = acacaacc$ liefert der Algorithmus das Ergebnis $6 = 1 + 3 + 2$, denn es gibt einen Treffer von aa, drei Treffer von ac, und zwei Treffer von ca.

Sie können dabei selbstverständlich Teile von Algorithmen aus der Vorlesung oder Übung nutzen, ohne diese nochmals beschreiben zu müssen. Sie müssen dann jeweils genau angeben, welchen Algorithmus bzw. welchen Teil des Algorithmus Sie meinen, welche Ergebnisse des Algorithmus (z.B. eine Liste oder ein Array mit bestimmten Werten) Sie für die Lösung dieses Problems nutzen möchten, und wieviel Laufzeit hierfür zu veranschlagen ist.

4.11. In welcher Relation steht die q-Wort Distanz zur Edit-Distanz bzgl. der Einheitskostenfunktion? Wie kann man diese Relation nutzen?

4.12. Seien U und V Mengen. Definieren Sie den Jaccard-Index $J(U, V)$ und beantworten die folgenden Fragen:

- Gibt es Mengen U und V, für die $J(U, V)$ undefiniert ist? Falls Ihre Antwort ja ist, dann geben Sie diese Mengen an. Falls die Antwort nein ist, dann begründen Sie Ihre Antwort.
- Welche Eigenschaften müssen die Mengen U und V besitzen, damit $J(U, V)$ minimal ist?
- Welche Eigenschaften müssen die Mengen U und V besitzen, damit $J(U, V)$ maximal ist?

4.13. Ein zentraler Begriff bei der Distanzschätzung mit dem Programm `mash` ist der Sketch einer Sequenz. Mit welcher Laufzeit, abhängig von der Sketch-Größe s, dem Parameter $q \in O(\log s)$ und der Sequenzlänge m kann ein Sketch berechnet werden? Begründen Sie Ihre Antwort.

4.14. Definieren Sie den Begriff Poisson-Verteilung für einen Parameter λ. Welche Eigen-

schaften müssen Ereignisse in einem Intervall (zeitlich oder geographisch) haben, damit die Wahrscheinlichkeiten des Auftretens von k Ereignissen durch eine Poisson-Verteilung bestimmt werden kann?

5 Signifikanzberechnung von Alignments

5.1. Gegeben seien zwei Sequenzen u und v. Was ist der E-value eines lokalen Alignments von u und v der Länge ℓ mit k Unterschieden (z.B. Mismatches und Indels). Hier ist keine genaue Formel gefragt, sondern eine allgemeine Definition.

5.2. Bei der Berechnung von E-values für exakte Matches wurde die Menge $M_{\max\geq\ell}$ betrachtet, die Menge der maximalen Treffer mit Mindestlänge ℓ in zwei Sequenzen u und v. Um den E-value für maximale exakte Treffer der Mindestlänge ℓ in zwei Sequenzen u und v zu bestimmen, wurde gezeigt, dass es eine andere Menge von Treffern gibt, die die gleiche Größe hat wie $M_{\max\geq\ell}$. Definieren Sie diese Menge und zeigen Sie, dass diese Menge die gleiche Größe hat wie $M_{\max\geq\ell}$. Hier ist kein formaler Beweis notwendig, sondern lediglich die Idee eines Beweises.

5.3. Wir betrachten eine Menge M der Größe n. Sei $k \leq n$. Wieviele Teilmengen von M mit genau k Elementen gibt es?

6 Fasta

6.1. Beim Fasta-Algorithmus wird im ersten Schritt für die Datenbanksequenz u und für die Anfragesequenz w sowie den Parameter q, der Fasta-Score $score_{\text{fasta}}(u, w)$ berechnet. Definieren Sie den Fasta-Score von u und w bzgl. q.

6.2. Beim Fasta-Algorithmus wird im ersten Schritt für die Datenbanksequenz u und die Anfragesequenz w sowie den Parameter q der Fasta-Score $score_{\text{fasta}}(u, w)$ berechnet. Für die effiziente Berechnung des Fasta-Scores wird die Anfragesequenz w zu einer Tabelle h_w vorverarbeitet. Geben Sie die Eigenschaften von h_w an, ohne zu beschreiben, wie h_w berechnet wird.

6.3. Beim Fasta-Algorithmus wird im ersten Schritt für die Datenbanksequenz u und die Anfragesequenz w sowie den Parameter q der Fasta-Score $score_{\text{fasta}}(u, w)$ berechnet. Für die effiziente Berechnung des Fasta-Scores wird, bei gegebener Alphabetgröße r, die Anfragesequenz w zu einer Tabelle h_w vorverarbeitet, so dass für alle c, $0 \leq c \leq r^{q-1}$ gilt:

$$h_w(c) = \{i \mid 1 \leq i \leq |w| - q + 1, c = \overline{w[i \ldots i+q-1]}\}$$

Nehmen Sie an, dass h_w bereits berechnet wurde. Beschreiben Sie kurz und präzise, wie man mit Hilfe von h_w, den Fasta-Score effizient berechnen kann.

7 Blast

7.1. In der Vorlesung wurden die wichtigsten Algorithmen des Programms Blast betrachtet. Sei u die Datenbanksequenz und w die Anfragesequenz. Seien außerdem die Scorefunktion σ und die Parameter k und q gegeben. Definieren Sie den Begriff Blast-Hit von u und w.

7.2. In der Vorlesung wurden die wichtigsten Algorithmen des Programms Blast betrachtet. Sei u die Datenbanksequenz und w die Anfragesequenz. Seien außerdem die Scorefunktion σ und die Parameter k und q gegeben. Zur Berechnung von Blast-Hits wurde der Begriff der k-Umgebung eingeführt. Definieren Sie den Begriff k-Umgebung von w.

7.3. Beschreiben Sie mit wenigen Sätzen, kurz und präzise die 2-Hit-Strategie von Blast. Worin besteht die X-drop-Erweiterung bei der Erweiterung ohne Gaps? Welche Rolle spielt dabei der *drop-off*-Parameter X_d?

8 PSSMs

8.1. Was ist eine positionsspezifische Score-Matrix (PSSM)? Woraus wird eine PSSM typischerweise abgeleitet?

8.2. Sei eine positionsspezifische Score-Matrix (PSSM) der Länge m gegeben, sowie ein Schwellwert th und eine Sequenz S. Was bedeutet in diesem Kontext der Begriff PSSM-Score und PSSM-Treffer? Mit welcher Laufzeit lassen sich alle Treffer einer PSSM der Länge m in einer Sequenz der Länge n finden?

9 Alignment in linearem Platz

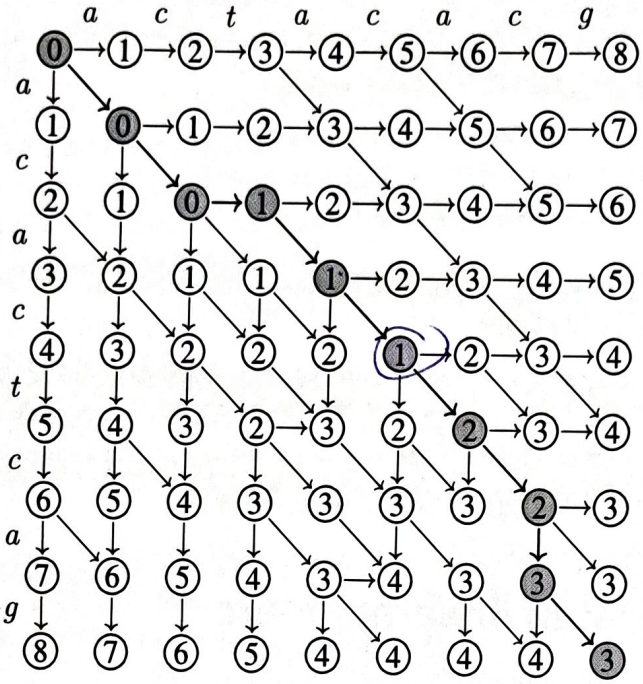
9.1. In der Vorlesung wurde ein Algorithmus zur Berechnung eines optimalen Alignments von Sequenzen u und v beschrieben, der nur linearen Speicherplatz benötigt. Dieser Algorithmus berechnet eine Tabelle C von Kreuzungspunkten. Definieren Sie kurz und präzise mit Hilfe des Edit-Graphen $G(u, v)$ von u und v die Eigenschaften der Kreuzungspunkte $C(j)$ für $0 \leq j \leq n$ und $n = |v|$. Geben Sie ausserdem an, in welchem Wertebereich die Kreuzungspunkte liegen und in welcher Beziehung zwei direkt aufeinander folgende Kreuzungspunkte stehen.

9.2. Der Algorithmus zur Berechnung eines Alignments in linearem Platz basiert auf einer rekursiven Strategie, die viele Einträge der DP-Matrix immer wieder neu berechnet. Was ist die maximale Rekursionstiefe und welche Laufzeit hat der Algorithmus für zwei Sequenzen der Längen m bzw. n? Begründen Sie Ihre Angabe der Laufzeit, indem Sie bestimmen, wieviel Matrix-Einträge der Algorithmus in Rekursionstiefe k berechnet und summieren Sie dann bis zur maximalen Rekursionstiefe.

9.3. Hier sehen Sie die Knoten des Edit-Graphen für die Sequenzen $u = acactcag$ und

$v = actacacg$ und alle minimierenden Kanten bzgl. der Einheitskostenfunktion. Der Pfad vom Knoten $(0, 0)$ nach $(8, 8)$, bestehend aus den grau gezeichneten Knoten, die durch fett gezeichnete Kanten verbunden sind, repräsentiert ein optimales Alignment der beiden Sequenzen. Geben Sie bitte folgendes an:

- den gekennzeichneten Pfad als Folge seiner Knoten,
- alle Listen von Kreuzungspunkten (dieser Begriff wurde im Kontext des Linear Space Alignment-Algorithmus eingeführt), die sich aus dem gekennzeichneten Pfad ableiten lassen,
- das Alignment mit den beiden Sequenzen in zwei Zeilen und dem Zeichen – zur Darstellung von Lücken.



10 MSAs

10.1. Geben Sie die in der Vorlesung definierten Eigenschaften an, die ein multiples Alignment von $k \geq 2$ Sequenzen $s_1, \ldots, s_k$ erfüllen muss.

10.2. In der Vorlesung wurden zwei Modelle zur Bewertung von multiplen Alignments betrachtet. Beschreiben Sie kurz und präzise das Konsensus Kostenmodell.

10.3. In der Vorlesung wurden zwei Modelle zur Bewertung von multiplen Alignments betrachtet. Beschreiben Sie kurz und präzise das Sum-of-Pairs Kostenmodell.

10.4. In der Vorlesung wurde gezeigt, wie man die Technik der dynamischen Programmierung zur Berechnung optimaler paarweiser Alignments auf $k > 2$ Sequenzen verallgemeinern kann. Dazu wird eine Matrix M benötigt. Welche Eigenschaften hat diese

Matrix, wenn ein multiples Alignment von k-Sequenzen $s_1, s_2, \ldots, s_k$ der Längen n_1, $n_2, \ldots, n_k$ berechnet werden soll. Hier ist nicht nach der Rekurrenz gefragt.

10.5. Wir betrachten das Problem der Berechnung eines multiplen Alignments für Sequenzen $s_1, s_2, \ldots, s_k$ der Längen $n_1, n_2, \ldots, n_k$. Dazu wird eine k-dimensionale Matrix M berechnet. Wieviele Einträge hat M und mit welcher Laufzeit kann M berechnet werden.

10.6. Beschreiben Sie kurz und präzise das Stern-Alignment-Verfahren für die Berechnung von multiplen Sequenzalignments einer Sequenzmenge S.

10.7. In der Vorlesung wurde das Stern-Alignment-Verfahren für die Berechnung von multiplen Sequenzalignments betrachtet. Geben Sie eine möglichst kleine obere Schranke für den Sum-of-Pair Score des Stern-Alignments relativ zum Sum-of-Pair Score des optimalen MSAs an.

10.8. Wie kann das in der Vorlesung vorgestellte Verfahren zur Berechnung optimaler multipler Sequenzalignments im Allgemeinen mit heuristischen Verfahren zur Berechnung von MSAs (z.B. dem Stern-Alignment) kombinieren, um es zu beschleunigen?

10.9. Worin bestehen die Vorteile und Nachteile von Verfahren zum iterativen multiplen Alignment?

11 Phylogenie

11.1. Beschreiben Sie kurz und präzise die UPGMA-Methode zur Rekonstruktion gewurzelter phylogenetischer Bäume.

11.2. In der Vorlesung wurde die UPGMA-Methode zur Rekonstruktion von phylogenetischen Bäumen für eine Menge X von Taxa beschrieben. Welche Voraussetzungen muss die Distanzfunktion d erfüllen, damit die UPGMA-Methode den korrekten gewurzelten phylogenetischen Baum für X rekonstruiert. Definieren Sie diese Eigenschaften von d.

11.3. Wir betrachten das UPGMA-Verfahren zum Rekonstruieren gewurzelter phylogenetischer Bäume für eine Menge X von Taxa. Wir nehmen an, dass es eine Distanzfunktion d gibt, die allen Paaren (x, y) von Taxa einen nicht-negativen Distanzwert zuweist. Seien C_i und C_j zwei disjunkte Teilmengen von X. Definieren Sie die Distanz $d(C_i, C_j)$.

11.4. Beschreiben Sie kurz, aber präzise die Methode des Neighbor-Joining zur Rekonstruktion ungewurzelter phylogenetischer Bäume.

GSA

1 Das Modell der Edit-Distanz

1.1 Ähnlichkeit bei der Analyse biologischer Sequenzen?

- similar sequence → same or similar meaning/effect
  eg: genes
- bei der Ermittlung der Phylogenetischen Abstammung

1.2 am häufigsten verwendetes Distanz Modell.

Das edit Distanz model: Bestimmung der edit distanz und alle optimalen Alignments. Kosten des optimalen Alignments von zwei Sequenzen u & v.

1.3 Edit-Operation, Alignment und Alignment von zwei Sequenzen u und v.

- Edit-Operation ist ein Paar

$$(\alpha, \beta) \in (A \cup \{\varepsilon\}) \times (A \cup \{\varepsilon\}) \setminus \{(\varepsilon, \varepsilon)\}$$

$\alpha \to \varepsilon$: deletion of the character $\alpha \in A$

$\varepsilon \to \beta$: insertion of the character $\beta \in A$

$\alpha \to \beta$: replacement of the character $\alpha \in A$ by character $\beta \in A$

- Alignment von zwei Sequenzen u und v:

is a sequence: $(\alpha_1 \to \beta_1, \ldots, \alpha_k \to \beta_k)$

of edit operations s.t.:

$$u = \alpha_1 \ldots \alpha_k \quad \& \quad v = \beta_1 \ldots \beta_k$$

- Alignment is a sequence of edit operations

GSA

1 Das Modell der Edit-Distanz

1.4 Anzahl aller Alignments zweier Sequenzen.

$$Aligns(m,n) = \begin{cases} 1 & , \text{if } m=0 \text{ or } n=0 \\ Aligns(m-1,n) + \\ Aligns(m-1,n-1) + \\ Aligns(m,n-1) & , \text{else} \end{cases}$$

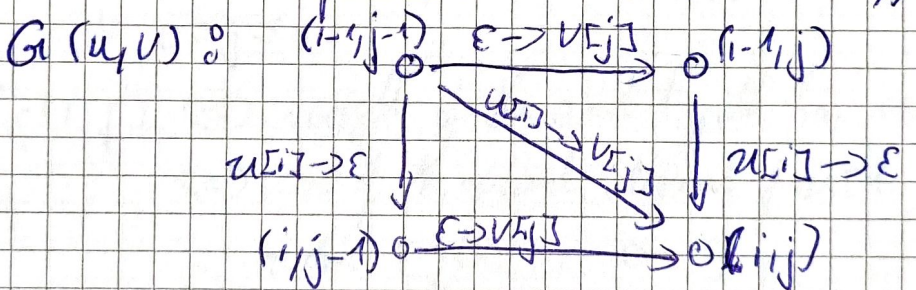
1.5 Menge alles alignments zweier Sequenzen der Länge m und n in $\mathcal{O}(mn)$ Speicherplatz?

~~Inside~~ In einer Edit Matrix E_S der größe $(m+1)\times(n+1)$. $E_S[i,j]$ beinhaltet die kosten des optimalen Alignment. # von $u[1...i]$ und $v[1...j]$ und einen Bit Vector der die Richtung des #

~~In einer Edit Matrix E_S der größe $(m+1)\times(n+1)$~~

In einem Edit Grapher der Form: // Edit-Matrix!

$G(u,v)$:



1.6 Datenstruktur, effiziente Speicherung Alignment.

Todo: siehe Übung!

Eine liste A-list = [X1 y1 ... Xk yk]

$X \in \{D, I, R\}$ $y \triangleq$ # aufeinander folgender X

und die sequenzen u und v.

eg: [R1 I2 D3 R2] u = ... v = ...

Quiz

1.7 Was ist eine Kostenfunktion im Kontext des Edit-Distanz Modells?

Die $edist_\delta(u,v)$ ist von der Kostenfunktion δ abhängig, welche jeder Edit-~~Distanz~~ Operation einen Wert zuteilt. Sie bestimmt wie die Kosten für ein Alignment berechnet werden.

1.8 $\delta(A)$, lineare GAP kosten.

$$\delta(A) = \sum_{i=1}^{h} \delta(\alpha_i \to \beta_i)$$

1.9 Def $edist_\delta(u,v)$, d.h.

$edist_\delta(u,v) = \min\{\delta(A) \mid A \text{ is an aligment of } u \text{ and } v\}$

1.10 optimales Alignment

An Alignment is optimal if $\delta(A) = edist_\delta(u,v)$

1.11 Welche Eigenschaften hat $E_\delta(i,j)$ $\forall i,j$ mit $0 \le i \le n$ & $0 \le j \le n$.

$E_\delta(i,j) \Leftrightarrow$ weight of the shortest path from $(0,0)$ to (i,j) in $G(u,v)$

$\Leftrightarrow$ cost of the optimal aligment of $u[1..i]$ and $v[1..j]$,

wobei $E_\delta(0,0) = 0$.

1.12 ~~Edit~~ $E_\delta(i,j)$, $j>0$ & $i>0$

$$E_\delta(i,j) = \min \begin{cases} E_\delta(i-1,j) + \delta(u[i] \to \varepsilon), \\ E_\delta(i,j-1) + \delta(\varepsilon \to v[j]), \\ E_\delta(i-1,j-1) + \delta(u[i] \to v[j]), \end{cases} \quad j>0 \;\&\; i>0$$

GSA

1.13 Es für eg:

$E_\delta(i,j)$		... $j=5$	a	t 6
i	0	5	←	6
g	1	4	←	5
c	2	4	← or ↖	5
a	3	3	← ↖	4
c	4	4	↖	4
t	5	4		4

R > D > I

1.14 Laufzeit / Speicherbedarf : Distance-only ($edist_\delta(v,v)$)

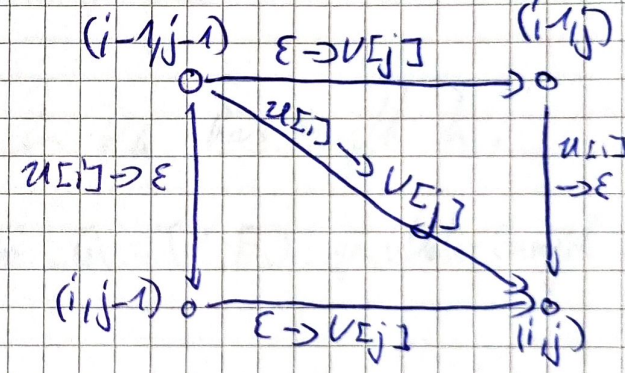
Laufzeit : $O(mn)$ Speicherbedarf : $O(\min\{m,n\})$

1.15 Edit Graph

edit graph $G_E(v,v)$

Nodes: (i,j), $0 \le i \le m$, $0 \le j \le n$!

Edges: Edit operations:



D edge: $1 \le i \le m$, $0 \le j \le n$:

$(i-1,j) \xrightarrow{u[i]\to\varepsilon} (i,j)$

I edge: $0 \le i \le m$, $1 \le j \le n$:

$(i,j-1) \xrightarrow{\varepsilon\to v[j]} (i,j)$

R edge: $1 \le i \le m$, $1 \le j \le n$:

$(i-1,j-1) \xrightarrow{u[i]\to v[j]} (i,j)$

1.16 Verfahren: optimales globales Alignment für zwei Vektoren aus der Matrix E§:

Ein Traceback Verfahren, dass die Minimumsenden Pfade von E§[m][n] bis E§[0][0].
Realisiert mit Hilfe von Bit Vektoren die, die Richtung der minimalen Pfade angibt.

1.17 Graph

Ein Graph besteht aus einer Menge von Knoten V und einer Menge von Kanten $E \subseteq V \times V$.

1.18 Path in $G = (V, E)$

Ein Path in einem Graphen G ist eine Sequenz von Knoten $v_0, v_1, \ldots, v_k$ für $k \geq 0$, sodass $\forall i, 0 \leq i \leq k-1$ $(v_i, v_{i+1}) \in E$.
A Path starts in v_0 and ends in v_k has length k.

1.19 Topologische Ordnung in $G = (V, E)$ gerichteter Graph

Topologische Ordnung $\prec$ of V:

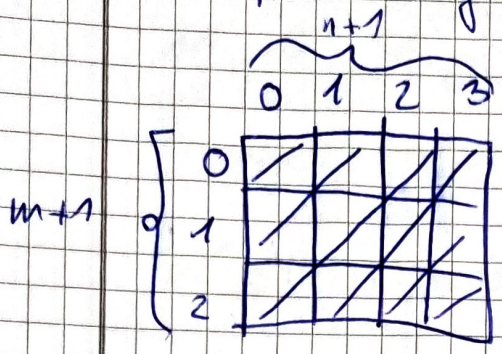
$$y \to x \in E \Rightarrow y \prec x$$

GSA

1.20

.1 # Antidiagonalen

Antidiagonalen = $m + n + 1$



.2 Eigenschaften aller Paare (i, j), die zu der gleichen Antidiagonalen a gehören.

Antidi: $(i', j')\ (i'+1, j'-1)\ \dots$

stopp bei $i' = m$ or $j = 0$

$i' + j' = a$ = # of Antidiagonale

.3 Von welcher Antidiagonalen in Eg hängen die Werte der Antidiagonale a für $a \geq 2$ ab?

	0	1	2	3
0	a_0	a_1	a_2	a_3
1	a_1	a_2	a_3	a_4
2	a_2	a_3	a_4	a_5

Die Werte von $a \geq 2$ hängen von den Antidiagonalen $a-1$ & $a-2$ ab.

GSA

2 Varianten des Edit-Distanz Modells

2.1 $\mathcal{L}_\sigma(i,j)$

~~$\mathcal{L}_\sigma(i,j) = \max\{\sigma(A) \mid A$ is an Alignment of a' substring of u and v' substring of v$\}$~~

$score_\sigma(u',v') = \max\{\sigma(A) \mid A$ is an alignment of u' and $v'\}$

$\mathcal{L}_\sigma(i,j) = \max\{score_\sigma(x,y) \mid x$ is suffix of $u[1..i]$ & y is suffix of $v[1..j]\}$

$\Rightarrow$ each entry (i,j) contains the max score for all pairs of suffixes of prefix $u[1...i]$ of u and prefix $v[1...j]$ of v.

Der maximale Eintrag in $\mathcal{L}_\sigma(i,j)$ ist $loc_\sigma(u,v)$.

2.2 $\mathcal{L}_\sigma(i,j)$

$$\mathcal{L}_\sigma(i,j) = \max\begin{cases} 0 \\ \mathcal{L}_\sigma(i-1,j) + \sigma(u[i] \to \varepsilon) \\ \mathcal{L}_\sigma(i,j-1) + \sigma(\varepsilon \to v[j]) \\ \mathcal{L}_\sigma(i-1,j-1) + \sigma(u[i] \to v[j]) \end{cases}$$

2.3

↳ II) Berechnen den maximalen Eintrag von $\mathcal{L}_\sigma(i,j)$ in $\mathcal{L}_\sigma$.

III) Traceback von (i,j) über maximierenden Pfad bis zu einem Eintrag (i',j') mit $\mathcal{L}_\sigma(i',j') = 0$, zur Bestimmung von optimal local alignment.

2.4 Approximater Treffer eines Musters p in Text t.

Ist ein Substring w von t:

$$edist_\delta(p, w) \leq k$$

2.5 Solving approximate string matching problem

.1 How?

↳ Berechne $(m+1) \times (n+1)$-table D_δ

$0 \leq i \leq m,\ 0 \leq j \leq n$:

$$D_\delta(i,j) = \min\{edist_\delta(p[1..i], w) \mid w \text{ is a suffix of } t[1..j]\}$$

↳ ~~Berechne~~ Bestimme alle j mit $D_\delta(m,j) \leq k$

.2 Runtime?

$O(mn)$

Space $O(m)$ without trace back!

.3 vs E_δ?

D_δ : Erste Zeile ist gleich 0

E_δ : $E_\delta(0, j-1) + \delta(\varepsilon \to U[j])$ // Depends on Insertion cost

2.6 Wie lang kann ein approximativer Treffer höchstens werden, δ und Kosten?

Deletions beeinflussen die Länge des Treffers nicht

Es gibt höchstens m Replacements. Insertions kosten $cost \geq \min\{\delta(\varepsilon \to b) \mid b \in A\} > 0$.

At least Insertion costs of $\gamma \cdot \min\{\delta(\varepsilon \to b) \mid b \in A\} \overset{!}{\leq} k$

$\Leftrightarrow \quad \gamma \leq \left\lfloor \frac{k}{\min\{\delta(\varepsilon \to b) \mid A \in A\}} \right\rfloor$, Max γ erlaubt!

$\Rightarrow$ |approximativer Treffer| $= O(m + k)$

2.7 Überlappungsproblem bei der Sequenzassemblierung.

↳ für ein paar f & f' von Reads

= Entscheiden ob f & f' überlappen und wenn das der Fall ist den Overlap A von f & f' zu berechnen mit $S(A) \leq k$

3 Spezielle Graphkosten

3.1 Lineare Graphkosten zu allgemeinen Gap-Kostenmodell:

define: $w \in A^+$:

$$\delta^*(w \to \varepsilon) = \delta^*(\varepsilon \to w) = f^*(|w|)$$

$f^*(|w|)$ = ~~[illegible]~~ // Die Deletionen gemacht

$= |w| * \delta(\text{Indel})$ // |w| D's, / I's werden gemacht auf einmal

$= a \times$ // linear!

~~[illegible]~~

↳ Der Rest kann dann auf das allgemeine Graphkostenmodell übertragen werden

3.2 Laufzeit (edist allgemeine Gap-Kostenmodell):

$$\mathcal{O}(m\,n\,(n+m))$$

3.3 Affine Gap-Kosten

Für jeden Gap ~~werden~~ wird die Summe für die Kosten für jedes Symbol im Gap und Gapinitiierung Kosten in Betracht gezogen.

indiemp kosten nur einmal.

$w \in A^+$:

$$\delta_{affine}(w \to \varepsilon) = g + \sum_{i=1}^{|w|} \delta(w[i] \to \varepsilon)$$

↳

$$\delta_{affine}(\varepsilon \to w) = g + \sum_{i=1}^{|w|} \delta(\varepsilon \to v[j])$$

1 × Indiemp Kosten dann Gap Extension Kosten

4 Sequenzvergleich ohne Alignments

4.1 Zerlegung = Partition

Eine Zerlegung ψ von v bzgl. u ist eine Sequenz $(w_1, c_1, \ldots, w_r, c_r, w_{r+1})$ von Substrings $w_1, \ldots, w_{r+1}$ von u und Characters $c_1, \ldots, c_r$ so dass $v = w_1 c_1 \ldots w_r c_r w_{r+1}$.

$mmdist(u,v)$ ist die Größe der minimalen Partition von v bzgl u. $|\psi| = r$

4.2 Maximal Matches Modell

.1 Keine Metrik

$V = cba\ abd\ cda$, $u = abcba$

$\psi_{lr}(v,u) = (cba, a, b, cda) \rightarrow mmdist(v,u) = 2$

$\psi_{lr}(u,v) = (ab, c, ba) \rightarrow mmdist(u,v) = 1$

↳ not symmetric !

.2 $mmm(u,v)$

$$mmm(u,v) = \log_2((mmdist(u,v)+1) \cdot (mmdist(v,u)+1))$$

4.3 Laufzeit / Speicher : $mmdist(u,v)$

Time: $\mathcal{O}(|A|(m+n))$, Space: $\mathcal{O}(m)$

$\mathcal{O}(|A|(|u|+|v|))$ $\quad$ $\mathcal{O}(|u|)$

4.4

$$\text{mmdist}(v,u) \leq \text{edist}_S(v,u)$$

- lower bound for $\text{edist}_S(v,u)$
- used as filter in contexts, where the unit edist_S is of interest only below some threshold h.
- ignore pairs where $\text{mmdist}(v,u) > h$;

4.5 q-Wort

.1 q-Wort Profile von u

↳ Es bildet alle $w \in A^q$ auf einen ineger ab, so dass $P_{u,q}(w) = \#$ an verschiedenen positionen in u an denen w vorkommt.

.2

$$q\text{gdist}(u,v) = \sum_{w \in A^q} |P_{u,q}(w) - P_{v,q}(w)|$$

4.6 No Metrik

$A = \{a,c\}$

$u = acaa$ $\quad aa \mapsto 1, ac \mapsto 1, ca \mapsto 1, cc \mapsto 0$

$v = aaca$ $\quad aa \mapsto 1, ac \mapsto 1, ca \mapsto 1, cc \mapsto 0$

$q\text{dist}(u,v) = 0 \not\Rightarrow u = v$ ↯

4.7 q-Worte codieren

~~(a) lexicographisch sortiertes Alphabet A~~

1) sortiertes Alphabet $A = \{a_1, .., a_r\}$, $a_1 < \dots < a_r$

2) Codiere chars: $\overline{a_l} = l - 1$

3) codiere q-Worte: $\overline{w} = \sum_{i=1}^{q} \overline{w[i]} \, r^{q-i}$
for all $w \in A^q$

4.8 Welche Eigenschaften dieser Codierung nutzt man aus, um nachher die Codes aller q-Worte mit Sequenz effizient zu berechnen?
Aufeinander folgende q-Worter mit Sequenzen können in constanter Zeit über bestimmt werden:

$$\overline{xc} = (\overline{ax} - \overline{a} r^{q-1}) r + \overline{c}$$

4.9 qgdist (u, v):
Time: $O(m+n+r^q)$, Space: $O(m+n+r^q)$

4.10 Daw Algorithmus.

1) Bestimme q-gram Profiles of u and v in two arrays τ_u and τ_v:

$$\tau_u[\bar{w}] = P_{u,q}(w) \;\&\; \tau_v[\bar{w}] = P_{v,q}(w)$$

$\forall w \in A^q$

- initialisieren von den arrays: $O(r^q)$ time
- Compute the q-gram codes for u & v: $O(m+n)$ time
- In $O(r^q)$ space.

2) Bestimme $Z = \sum_{i=0}^{r^q-1} |\tau_u[\bar{w}_i] \cdot \tau_v[\bar{w}_i]|$

$O(1)$ Space, $O(r^q)$ time

↳ $O(m+n+r^q)$ time, $O(r^q)$ space

4.11 qgdist(u,v) & edist$_s(u,v)$

$$\text{qgdist}(u,v) / (2 \cdot q) \leq \text{edist}_s(u,v)$$

Lower bound for edist$_s$, can be used as filter if edist$_s(u,v) \geq k$, then stop pair with qgdist$(u,v) \geq 2k$

GuSA

4.12 Jaccard-Index $J(U,V)$

$$J(U,V) = \frac{|U \cap V|}{|U \cup V|}$$

- wenn $U = \emptyset = V \rightarrow J(U,V) = \frac{0}{0}$ ↯
- J minimal für $U \cap V = \emptyset$ → $J(U,V) = 0$ (disjunkt)
- J maximal für $U = V \rightarrow J(U,V) = 1$

4.13 $q \in O(\log s)$ calculate sketch for seq length m

$$O(m(q + \log s)) = O(m \log s)$$

- enumerate all q-grams of u: $O(m)$ time $O(q)$ space
- apply hash function on all q-grams: $O(mq)$
- use min heap to keep s smallest hash values: $O(m \log s)$ time

$$\Rightarrow O(m \log s)$$

4.14 Eigenschaften der Anwendung von Poisson?

- k # of times event occurs
- events occur independtly
- rate at which event occur is constant
- two events never occur at the same sub-interall

GSA

5 Signifikanzberechnung von Alignments

5.1 Was ist der E-value eines lokalen Alignments von u und v der Länge ℓ mit k Unterschieden?

Der E-value ist die Anzahl an lokalen Alignments der Länge $\ell' \leq \ell$ mit $k' \leq k$ Unterschieden, die ~~~~ man zwischen Random Sequencen von der gleichen Länge wie u & v erwarten würde.

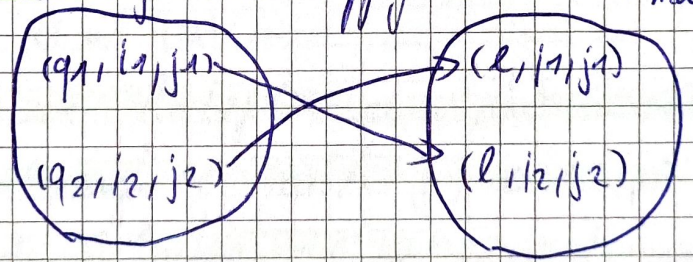
E-value = $|\{(\ell', i', j') \mid (\ell', i', j')$ is k'-LMA in $u' \& v'$, $\ell \leq \ell', k \leq k'\}|$

5.2

$$|M_{max \geq \ell}| = |M_{\ell max = \ell}|$$

↳ Menge der left-maximalen exakten Treffer der Länge $= \ell$.

- Construct bijektives Mapping zwischen $M_{max \geq \ell}$ & $M_{\ell max = \ell}$



$(q_k, i_k, j_k) \rightarrow (\ell_k, i_k, j_k)$ ~~~~ $q \geq \ell$

(ℓ_k, i, j) is ℓmax und $rmax$ of $q > \ell$

⇝ Show mapping is bijektive

5.3 Menge M der Größe n. $k \leq n$

Teilmenge von M mit k elementen.

$$\binom{\ell}{k} = \frac{\ell!}{k! (\ell - k)!}$$

6 Fasta

6.1 Fasta Score

$|w| = n$
$|r| = m$

$$score_{fasta} = \max \{ hotsp(r, w, d) \mid -m \leq d \leq n \}$$

$-m \leq d \leq n$:

$$hotsp(r, w, d) = |\{ (i,j) \mid 1 \leq i \leq m-q+1,\ 1 \leq j \leq n-q+1,\ j-i = d,\ r[i..i+q-1] = w[j..j+q-1] \}|$$

6.2 Properties of h_w

$\forall\ c,\ 0 \leq c \leq r^q - 1$

$h_w(c)$: positionen in w, an denen ein q-gram auftritt mit hat Code c.

$$= \{ i \mid 1 \leq i \leq |w| - q + 1,\ c = \overline{w[i..i+q-1]} \}$$

6.3 Calculate $score_{fasta}$ with h_w

Für ein r mit $|r| = m$

1) Initialisiere $hotsp(r, w, d) \leftarrow 0$.
2) gehe durch r mit j, $1 \leq j \leq m-q+1$
   2.1) Calculate c = score of current q-gram $r[j..j+q-1]$
   2.2) Für alle $i \in h_w(c)$ ~~calculate~~ increment $hotsp(r, w, j-i)$ ++
3) calculate

$$score_{fasta}(r, w) = \max \{ hotsp(r, w, d) \mid -m \leq d \leq n \}$$

7 Blast

7.1 Blast Hit

query w , $u \in S$

A blast hit is a pair (i,j) s.t.:

$$score_0(u[i \dots i+q-1], v[j \dots j+q-1]) \geq k$$

7.2 k-Environment

$$Env_k(w) = \{(s,j) \mid 1 \leq j \leq |w| - q + 1, score(s, w[j \dots j+q-1]) \geq k\}$$

7.3 Ungapped Extension

- Calculate Mid point (i,j)
- extension to the left: $u[i \dots 1 \dots i-1]$ & $w[1 \dots j-1]$ are compared $\leftarrow$
- extension to the right: $u[i \dots m]$ & $w[j \dots n]$ are compared $\rightarrow$

left: Calculate scores for $(u[i-l], w[j-l])$

$1 \leq l \leq \min\{i-1, j-1\}$

right: -11- for $(u[i+r], w[j+r])$

$0 \leq r \leq \min\{m-i, n-j\}$

keep track of total max score X_{max}.

~~Extend local alignment~~ till $X_{max} - X_d$ is reached!

sequence

↳ MSP

GSA

8 PSSMs

8.1 PSSM? How?

alphabet ↑ size of M ↑

PSSM is a function $M: A \times \{1,\dots,m\} \to \mathbb{R}$

↳ Jede Reihe gibt die Frequenz eines $a \in A$ für die Positionen $1 \dots m$ an. $|A| \times m$-matrix

Abgeleitet von ~~[illegible]~~ multiplen Alignment von einer Family eines, dass heißt von verwandten Sequenzen!

8.2 PSSM-Treffer & Laufzeit

PSSM-Treffer: eine Position j, $1 \le j \le |S| - m + 1$

s.t.: $score(S[j \dots j+m-1], M) \ge thr$

$O(kn)$ time, k is $\varnothing$ # of PSSM-positions per sequence die von der start-position aus wirklich ausgewertet werden

↳ worst case $O(mn)$

GSA

9 Alignment in linearer Platz

9.1 Eigenschaften von Kreuzungspunkten.

Wertebereich und zwei aufeinander folgende Cp.

- ein Crosspoint (i_0, j_0) liegt auf einem minimalen Pfad in Edit Graph $G_S(u,v)$ von $(0,0) \to (m,n)$
- Wertebereich $0 \le i_0 \le m$, $0 \le j_0 \le n$
  mit $(0,0)$ & (m,n) initial & final crosspoint
- Zeitaufeinander folgende cp's (i_0, j_0) & neue (i_0', j_0')
  $j_0 < j_0'$ & $i_0 \le i_0'$

9.2 Max Rekursionstiefe, Laufzeit $|U|=m$ & $|V|=n$

- Max Rekursionstiefe $O(\log_2 n)$
- $O(mn)$ Time:
  - → Evaluate $(m+1)(n+1)$ values in matrix E_S
  - → Evaluate $\frac{1}{2}(m+1)(n+1)$ values in R_{j_0}
  - → $\frac{3}{2}(m+1)(n+1)$ values
  - → After CP calculation: evaluate $(i_0+1) \times (j_0+1)$ matrix + half ib size
    $(m-i_0) \times (n-j_0)$ matrix

Ein Schritt ⇒ $\in \mathcal{O}\left(\frac{1}{2}\right) \frac{3}{2}(m+1)(n+1)$ # of matrix values computed

→ Using quadratic series: $\le 3(m+1)(n+1)$

$$\sum_{i=0}^{p_2} \frac{1}{2^i} pq = \sum_{i_0}^{q} \frac{1}{2^i} \cdot \frac{3}{2}(m+1)(n+1) \le 3(m+1)(n+1)$$

GSA

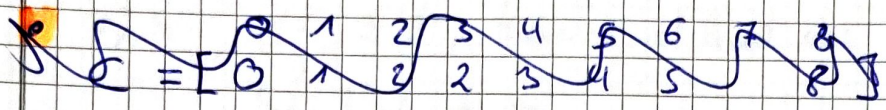
9.3 siehe Blatt

$u = acactcag$, $v = acacacg$

- Min Path: i, j

$(0,0) \to (1,1) \to (2,2) \to (2,3) \to (3,4)$

$\to (4,5) \to (5,6) \to (6,7) \to (7,7) \to (8,8)$



- Alignment:

$$\begin{pmatrix} a & c & - & a & t & c & a & g \\ a & c & t & a & c & a & c & - \end{pmatrix}$$

- Alle Listen von Kreuzpunkten:

	0	1	2	3	4	5	6	7	8	
C = [	0	1	2	2	3	4	5	6	8]	$\downarrow i$

$\downarrow j$

oder

	0	1	2	3	4	5	6	7	8	
C = [	0	1	2	2	3	4	5	7	8]	$\downarrow i$

$\downarrow j$

GSA

10 MSAs

10.1

MSA $s'_1, \dots, s'_k$, $k \geq 2$

1) $\ell \geq 1$ s.t. $s' \in (A \cup \{-\})^{\ell}$, $\forall i, 1 \leq i \leq k$

2) delgap $(s'_i) = s_i$, $\forall i, 1 \leq i \leq k$

3) $\forall j, 1 \leq j \leq \ell$, there is at least one i, $1 \leq i \leq k$, $s'_i[j] \neq -$

10.2 Consensus

1) Definition Kosten: $count(j, a) = |\{i \mid 1 \leq i \leq k, s'_i[j] = a\}|$

2) Bestimme Consensus:

↳ Sequenz s.t. $count(j, cons[j]) \geq count(j, a)$, $\forall a \in A$

↳ Strg für den ~~ca~~ am count am ~~häufigsten ist für alle~~ der dass am häufigsten ist an jeder position

3) Distance:

$$dist(cons, MSA) = \sum_{j=1}^{\ell} (k - count(j, cons[j]))$$

$$= conscost(MSA)$$

↳ soll minimiert werden!

GSA

10.3 Sum-of-Pairs Kostenmodell

↳ Calculate the sum of scores of all pairs of the chars and gaps in each colum of the MSA

$$\delta_{SP}(s_1'[j], \dots, s_k'[j]) = \sum_{i=1}^{k-1} \sum_{r=i+1}^{k} \delta(s_i'[j] \to s_r'[j])$$

SP-alignment kosten:

$$\delta_{SP}(s_1', \dots, s_k') = \sum_{j=1}^{l} \delta_{SP}(s_1'[j], \dots, s_k'[j])$$

↑ minimize this

10.4

$M_{\vec{n}}$ $(n_1+1) \times \dots \times (n_k+1)$ - Matrix so, dass

$(q_1, \dots, q_k) \in [0, n_1] \times \dots \times [0, n_k]$

↳ $M(q_1, \dots, q_k)$ is the score of the optimal MSA for $s_1[1..q_1], \dots, s_k[1..q_k]$

10.5

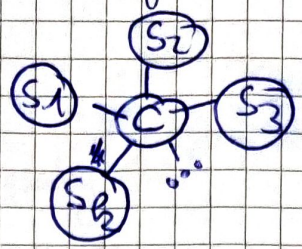
M hat $z = \prod_{i=1}^{k} (n_i + 1)$ Einträge

$\mathcal{O}(z \cdot f(k) \cdot 2^k)$

↳ depends on scoring function!

GSA

10.6 Stern-Alignment Verfahren



Build Alignment Tree

Edge optimal paarwise Alignment

1) Identify the center sequence
2) Construct from the optimal paarwise alignments a MSA, that is compatible with these alignments

10.7

$$\delta_{SP}(MSA_{Stern}) \leq (2 - \frac{2}{k})\, \delta_{opt\text{-}sp}(s_1, \ldots, s_k)$$

↑ Stern Alignment

↑ optimales SP MSA

10.8

use $\delta_{SP}(MSA_{Stern})$ as upper bound for $\delta_{opt\text{-}sp}$.

$M(entry) = d > \delta_{SP}(MSA_{Stern}) \rightarrow d > \delta_{opt\text{-}sp}$ Stopp!

~~($d \leq \delta_{SP}(MSA_{Stern})$ stopp! zu klein~~

→ Do not compute entry in M larger than $\delta_{SP}(MSA_{Stern})$

MSA

10.9 +/- Iterative Verfahren

(+) fast algorithms for many & long seq's

(+) Conservation in subfamilies that are aligned better
↳ high quality MSA

(-) Errors in early step "once a gap always a gap"

(-) no well defined objective function (needs to be optimized)

(-) guide tree might be bias

11 Phylogenie

11.1 UPGMA-Methode

1) Start with nodes x_i representing a Cluster C_i and place it a height zero

2) Determine pair (i,j) (C_i, C_j) with $d(C_i, C_j)$ is minimal
   Create a new Cluster $C_k = C_i \cup C_j$
   Compute $d(C_k, C_l)$ for all Clusters C_l
   New node k with childs i,j at height $\frac{1}{2} d(C_i, C_j)$

3) two clusters place root at $\frac{1}{2} d(C_i, C_j)$

11.2 Eigenschaften von d für UPGMA

1) d is additive w.r.t T_X
   ~~d is additive~~
   if it was obtained by adding up edge lengths on paths between the leaves of T_X

2) d is ultrametric, if and only if every leaf in T_X has the same dist to the root
   if $\forall (x,y,z) \in X \times X \times X$
   the dists $d(x,y), d(x,z), d(y,z)$ are of:
   1) all are the same
   2) two are equal and one is smaller

GrSA

11.3 $d(C_i, C_j)$

$$d(C_i, C_j) = \frac{1}{|C_i| |C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

11.4 Neighbor Joining Method

Introduce: $N_{ij} = d(i,j) - (r_i + r_j)$, $r_i = \frac{1}{n-2} \sum_{x \in L} d(i,x)$

1) Leafs → Node in T & $L \leftarrow T$

2) Compute N and find pair i,j s.t. N_{ij} is min.
   - Add new node k and update dists for $\forall m \in L \setminus \{i,j\}$: $d(k,m)$
   - Add k to T and construct edges to i,j.
   - Remove i & j from L and add k to L

3) Four nodes add edge i,j with $d(i,j)$